CANEI SUBIRATS, S.L. · PILOT

The ERP landscape, and exactly what to do next

Where the system stands today, why you and Ignacio are not yet seeing the same data, and the six steps that fix it. No Cloudflare. No domain transfer. No laptop.

The one sentence that explains everything

Your server works. The app on your phone has never been connected to it — it loads a static website that saves everything into that phone's own storage.

Status at a glance

Part of the system	State	What that means
Server — Hetzner Cloud, 178.105.10.156	RUNNING	Application, database and nightly backups all up.
Database — PostgreSQL	VERIFIED	Customers written from the app land here and read back.
Deploying code — GitHub → server	WORKING	Both of you can ship. Tests now gate the release.
Login — accounts for you and Ignacio	BUILT	Tested end to end. Needs switching on (steps 1–4).
Reachable from a phone	NOT YET	Needs steps 3–5. Everything for it is written.
Phone apps — iOS & Android	WRONG TARGET	Point at a static website, not your server. Step 6.
Shared calendar screen	NOT SHARED	Data and commands work; the screen still saves locally.
Restore from backup	UNTESTED	Backups are taken. Restoring has never been rehearsed.

Read this before the steps. Steps 1 to 5 are done once and take about half an hour. Step 4 is the only one that exposes the server to the internet, and it refuses to run until the login is genuinely working — so the order matters and the system enforces it.

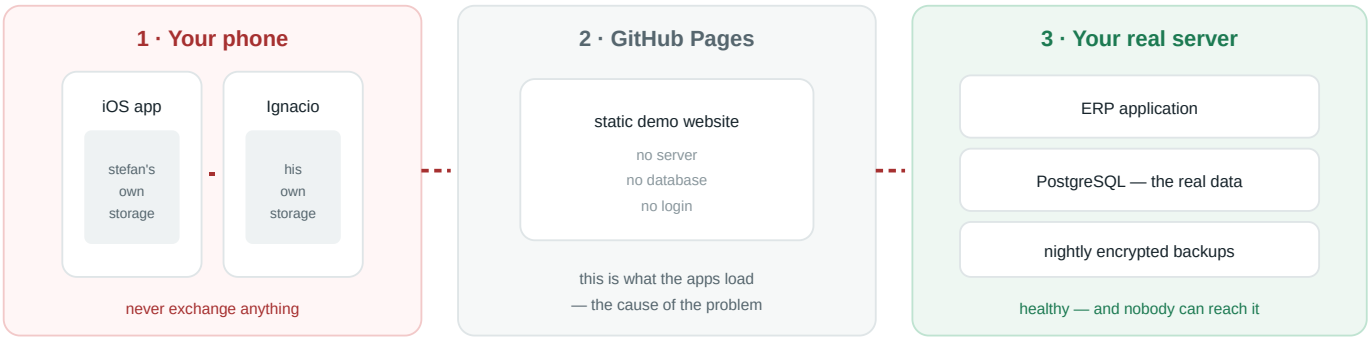
The six steps, at a glance

Steps 1–5 make it work today. Step 6 is polish — the browser on your phone is the same application.

THE LANDSCAPE

Today: three separate islands

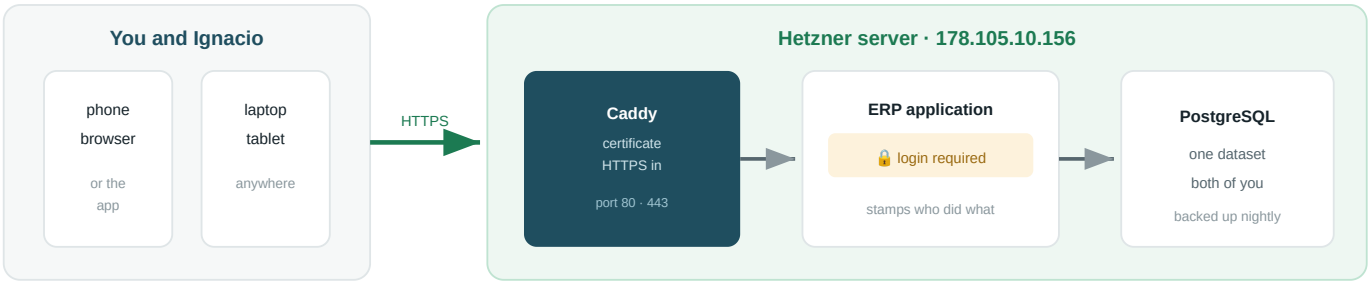
Data does not move between these. That is the whole problem.



✕ **no connection anywhere along this line**

The server is only reachable through an SSH tunnel from one Mac, because it has had no login of its own.

After the six steps: one system



✓ **Stefan enters a customer → Ignacio sees it. Ignacio adds a task → Stefan sees it.**

Both names recorded. Already proven with two signed-in sessions against a real database — see page 6.

Where each kind of data actually lives

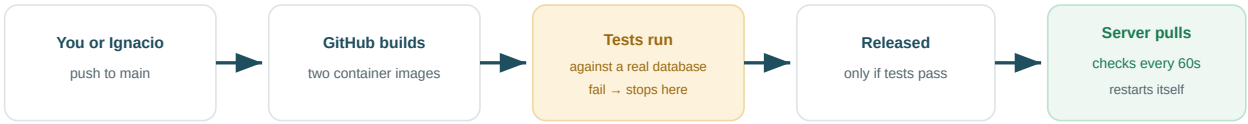
What you enter	Today	After the six steps
Customers & suppliers	on that device in the phone app	shared — already proven working
Money in and out	on that device	shared
Site progress, change orders	on that device	shared
Calendar tasks	on that device	half — the data is shared, the calendar *screen* still saves locally until it is moved
Quotes, contracts, invoices	on that device	not yet — the server does not accept these operations yet

The two amber rows are the honest remaining gap, and they are separate from anything in this document's six steps. Neither blocks the pilot: customers, money and site progress are the parts you need shared first.

HOW WORK REACHES THE SERVER

Deploying code — this already works, for both of you

Neither of you needs a key, a password or a laptop to ship a change. You push; the server collects it. Nothing ever logs into the machine.



Total: roughly four minutes from pushing to running. Only the `main` branch releases — a feature branch is built and tested but not shipped.

Fixed this week: the release used to happen BEFORE the tests finished. It no longer does.

Running the server itself — from a browser, no laptop

Health checks and backups now run from a button. One-time setup, done on your phone if you like.

Where	What
GitHub → Settings → Secrets and variables → Actions	Add two secrets, once: `HCLOUD_TOKEN` (your Hetzner API token) and `SERVER_SSH_KEY` (the whole private key file from `ops/.provisioned/id_ed25519`).
GitHub → Settings → Environments → production	Add yourself as a **required reviewer**. Then every ops run waits for your approval — worth doing, because anyone with write access to the repository can otherwise use those secrets.
GitHub → Actions → "Ops (run from a browser)"	Run workflow → choose `status`, `backup-now` or `list-ssh-allowed`. Read the log: every line is a ✓, a ! or a ✗, and the ones that are not ✓ say what to do.

Two people, one server. The old script set the SSH allow-list to exactly one address, so whoever ran it second locked the first one out — silently. Use `./ops/ssh-allow.sh add` instead: it changes one entry and leaves everyone else alone. If you ever do lock yourself out, Hetzner's web console reaches the machine anyway: **console.hetzner.com → the server → Console**.

Who does what, once the pilot is on

Task	Who	Where
Use the ERP day to day	**You and Ignacio**	The address from step 5, any device, after signing in.
Ship a code change	**You and Ignacio**	Push to `main`. Tests gate it; the server collects it within a minute.
Check the server is healthy	**You and Ignacio**	GitHub → Actions → Ops → `status`. No key needed.
Add or remove a person	**You**	Edit `ERP_USERS` in the server's `.env`, then restart.
Open or close internet access	**You**	`./ops/open-web.sh` or `--close`.
Backups, certificate renewal	**Automatic**	Nightly at 02:30 UTC, encrypted. Nothing to remember.

Keep the last two to yourself for now. Both mean editing the server's `.env`, which is also where the database passwords live — the difference between a colleague having an account and a colleague having the keys.

WHAT TO DO · PART ONE

Turning the pilot on

Six steps, about half an hour. Steps 1–2 create the accounts, 3–4 put the ERP on the internet behind them, 5–6 get it onto the phones.

1 Make a password for each of you

WHERE — a terminal, in your clone of the repository

```
node apps/web/scripts/hash-password.mjs stefan@caneisubirats.com
node apps/web/scripts/hash-password.mjs ignacio@caneisubirats.com
```

It asks for the password twice without showing it, then prints one long line per person. Copy both lines. Nothing is sent anywhere — this runs entirely on your machine. Use at least 12 characters, from the password manager; the ERP will be on the internet and there is no lockout after repeated guesses yet.

2 Put the accounts on the server

WHERE — on the server, in the file `/opt/canei-erp/.env`

Get there with `ssh -i ops/.provisioned/id_ed25519 root@178.105.10.156`, then `nano /opt/canei-erp/.env`. Or use Hetzner's web console if you prefer not to use a terminal.

```
# the two lines you copied in step 1, comma-separated, all on one line
ERP_USERS="stefan@caneisubirats.com:scrypt$...,ignacio@caneisubirats.com:scrypt$..."

# run: head -c 48 /dev/urandom | base64   and paste the result here
SESSION_SECRET="..."

# your server's address with dots replaced by dashes
PUBLIC_HOSTNAME="178-105-10-156.sslip.io"
ACME_EMAIL="stefan@caneisubirats.com"
```

Set `ERP_USERS` and `SESSION_SECRET` both or neither. With only one of them the application refuses to start, on purpose — the alternative is that it keeps working while quietly recording everybody's work under a single name, which looks completely normal and is very hard to notice later.

3 Start the piece that handles HTTPS

WHERE — on the server, still in that SSH session

```
cd /opt/canei-erp
docker compose -f docker-compose.prod.yml --profile pilot up -d
```

This starts Caddy, which obtains a real certificate automatically. The certificate is issued for `178-105-10-156.sslip.io` — a free public naming service that turns your IP address into a proper name. That matters because iPhones refuse a self-signed certificate outright, so without a real name the app could never work. When your own domain transfers, you change `PUBLIC_HOSTNAME` and nothing else.

Do not skip ahead to step 4. It is the step that opens the server to the world. It checks first, on the machine, that a request with no login is actually turned away — not merely that you configured one. If the running application is older than the login, it refuses and opens nothing.

WHAT TO DO · PART TWO

Opening it up, and getting it on the phones

4 Open the door — WHERE — a terminal, in your clone (needs `ops/provision.conf`)

```
./ops/open-web.sh
```

Opens ports 80 and 443 on the Hetzner firewall — but only after confirming the login is real. Wait about a minute afterwards: the certificate is obtained on the first request.

To take it back off the internet at any time: `./ops/open-web.sh --close`. SSH is unaffected and nothing is lost — the address simply stops answering.

5 Check it from your phone — on mobile data — WHERE — the browser on your phone, wifi turned off

```
https://178-105-10-156.sslip.io
```

Turn wifi off so you are testing the real path in from outside, not the office network. You should get the Canei sign-in screen. Sign in, then have Ignacio do the same on his phone. Enter a customer on one and look for it on the other — that is the whole pilot, proven.

6 Point the phone apps at the server — WHERE — two files in the repository, then a new build

```
ios/CaneiSubirats/Support/Config.swift — line 20
static let baseURL = URL(string: "https://178-105-10-156.sslip.io/")!

android/app/src/main/kotlin/com/caneisubirats/app/MainActivity.kt — line 26
private val baseUrl = "https://178-105-10-156.sslip.io/"
```

Both currently load `stefangruber001.github.io`, which is the static demo site with no server behind it — the direct cause of the calendar not being shared. After changing these, a new TestFlight and Play build is needed. **Until then, use the address from step 5 in the phone's browser** — it is exactly the same application.

The ten-second test, whenever something looks unshared

What the address bar says	What that means
`stefangruber001.github.io/…`	LOCAL ONLY Saved on that device. Nobody else will ever see it, however convincingly it appears to save.
`178-105-10-156.sslip.io`	THE SERVER Shared, backed up, attributed to you.

If two people are looking at two different addresses, they are looking at two different datasets. This is the single rule that explains the calendar problem, and it will explain the next one too.

HONEST LIMITS

What you will and will not have

✓ Works after these steps

- Both of you sign in from any device, anywhere.
- One shared dataset. A customer entered by one is visible to the other immediately.
- Every change records **who** made it — verified with two separate sessions writing to a real database.
- Simultaneous edits are safe: the second save is refused with “somebody saved before you”, never a silent overwrite.
- Nightly encrypted backups continue.
- Either of you can ship code changes without touching the server.

✕ Still missing

- **A full project from lead to close.** The server accepts twelve operations — customers, money in and out, progress, change orders, tasks, the quarterly archive. Quotes, contracts and invoices are not among them.
- **A shared calendar screen.** The data and the operations now work; the screen itself still saves into the browser. Next piece of work.
- **A rehearsed restore.** Backups are taken; restoring has never been tried.
- **Password reset, lockout, two-factor.** Resetting a password means editing the server's `.env`. Fine for two people, wrong for a third.

Three risks worth knowing about

Risk	What it means, and what to do
No limit on password guessing	Nothing yet slows an automated attempt at the login form. Use long passwords from the manager. Adding rate limiting is a small, known piece of work.
The address depends on a free service	`sslip.io` turns your IP into a name. If it stopped, the address would stop resolving until it returned. **No data is at risk** — it is on your server and in your backups. Moving to `caneisubirats.com` removes this entirely.
A lost phone stays signed in	Sessions last eight hours and cannot be cancelled one at a time. To cut everyone off immediately, change `SESSION_SECRET` in the server's `.env` and restart — everybody signs in again.

What was actually tested, not assumed

Everything above was driven against a real build, a real PostgreSQL and the restricted database account the server uses: an anonymous browser is redirected to sign in and an anonymous API call is refused; a wrong password sets no session and never reveals which half was wrong; Stefan added a customer, Ignacio signed in separately and added a calendar task, and Stefan's session then saw Ignacio's task. The database recorded `stefan@… → addParty` and `ignacio@… → addTask`. Signing out closed it again.

That run also found a genuine bug worth mentioning: adding a task accepted the person's name and silently discarded it, so a task had no author anywhere — invisible with one operator, wrong the moment two people share a schedule. Fixed, with tests.

Full written detail lives in the repository: `docs/PILOT-WITHOUT-CLOUDFLARE.md` for these steps, `docs/OPS-WITHOUT-A-LAPTOP.md` for running the server from a browser, and `docs/WHY-THE-CALENDAR-IS-NOT-SHARED.md` for the calendar diagnosis.